

CoC 7e LLM Keeper — Persistent Campaign Engine Roadmap

Version: v2.8–v3.0 Master Plan

Baseline: v2.7.6 “The Spoiler Channel”

Date: July 20, 2026

Purpose: Expand the original v2.8 offload roadmap into a phased build plan for a true persistent campaign engine with an AI Keeper.

1. Core Doctrine

The engine owns truth. The model owns voice.

The Keeper LLM should feel like a human Keeper: atmospheric, reactive, socially intelligent, willing to improvise, and capable of interpreting strange player declarations. It should **not** be responsible for remembering where everyone is, how much ammunition remains, which door was destroyed, which NPC heard the gunshot, or whether a clue has already been discovered.

Those facts belong to the campaign engine.

The engine owns

- Character sheets and derived values
- HP, SAN, Luck, MP, wounds, insanity, and skill improvement
- Inventory, equipment, ammunition, containers, and item condition
- Character and NPC locations
- Valid movement and travel time
- Object state
- Room state
- Clue discovery state
- Front clocks and timeline events
- Noise, visibility, and sound propagation
- NPC objectives, awareness, routes, and relationships
- Known facts, false beliefs, rumors, and secrets
- Dice results and their consequences
- Save/load, replay, and campaign audit trails

The LLM owns

- Narration and atmosphere
- NPC dialogue and emotional performance
- Private perception and unreliable narration
- Interpretation of unusual or ambiguous declarations
- Social nuance
- Horror pacing within deterministic constraints
- Proposed consequences, proposed facts, and proposed narrative beats
- Creative complications that do not violate canonical state

The boundary

The LLM may say:

“Eleanor’s shot destroys the lock. The sound echoes through the chapel.”

The engine must already know whether:

- Eleanor has a loaded firearm
- The shot hits
- Ammunition is consumed
- The lock breaks
- The noise propagates
- NPCs hear it
- Front clocks advance
- The room state changes

The LLM turns the engine’s outcome into horror fiction. It does not decide the outcome behind the curtain.

2. Current Problem Statement

v2.7.6 already has several strong foundations:

- Deterministic dice and combat resolution
- Sanity engine
- Spatial graph and sound propagation
- Scenario-defined locations, NPCs, fronts, clues, and timeline
- Pending dice requests
- Meta-command inventory prototype
- Private/public narration hygiene
- Cost and latency instrumentation
- Provider abstraction

However, several systems are still prototypes rather than persistent truth layers:

1. **The LLM can still mutate canonical character state directly through `state_delta`.**
2. **Inventory is a list of display names, not a set of persistent item instances.**
3. **Equipped weapons are recreated from catalog templates, so ammunition and condition can reset.**
4. **Scenario timeline data is loaded and saved but not actively dispatched.**
5. **Front trigger definitions are not yet consumed by an engine trigger system.**
6. **Clue definitions are loaded but not yet attached to deterministic discovery rules.**
7. **Facts are not separated from what each character knows or believes.**
8. **NPCs have attitude text, but no persistent objective, route, awareness, or relationship model.**
9. **The prompt compiler still presents broad state directly rather than purpose-built Keeper, scene, and player views.**

The goal of this roadmap is to close those gaps before expanding into multiplayer.

3. Roadmap at a Glance

Phase	Target version	Name	Primary result
0	v2.7.6.1	Truth Firewall	LLM output becomes proposal-only; state corruption is blocked
1	v2.8.0	Canonical Items and Objects	Persistent inventory, ammunition, equipment, and room objects
2	v2.8.1	Room Truth and Offline Movement	Static room state, dynamic overlays, and zero-LLM travel
3	v2.8.2	Event and Trigger Engine	Timeline, front thresholds, noise, and object changes fire deterministically
4	v2.8.3	Facts and Knowledge	Canonical facts separated from per-character knowledge and belief
5	v2.8.4	Clues and Objectives	Clues are earned, tracked, connected, and used by campaign logic
6	v2.8.5	NPC Director	Persistent NPC objectives, awareness, routes, and relationships
7	v2.8.6	Action Grammar and Adjudication	High-confidence player actions resolve locally; ambiguity remains freeform
8	v2.8.7	Truth-to-Voice Prompt Compiler	LLM receives outcome packets, not raw world authority
9	v2.8.8	Persistence, Replay, and Audit	Campaigns can be reconstructed, inspected, and regression-tested
10	v2.8.9	Cost and Provider Hardening	Reasoning budgets, local-model fallback, and measured cost controls
11	v3.0	Multiplayer Campaign Server	Per-player views, scene pacing, event sync, and networked terminals

Critical path: Phase 0 → 1 → 2 → 3 → 4/5 → 6 → 7 → 8 → 9 → 10 → 11.

Phases 4 and 5 may be developed in parallel after Phase 3, but both must land before the NPC Director and action grammar become authoritative.

Phase 0 — Truth Firewall

Target: v2.7.6.1

Theme: Stop canonical state corruption before building more state.

Goal

The LLM must no longer directly overwrite engine-owned truth. Its output becomes a set of narration and proposal fields that the engine validates.

This is the first phase because every later system depends on trusting the world state. If the model can silently rewrite HP, SAN, location, inventory, or NPC condition, persistent campaign logic cannot be reliable.

Primary modules

- New: `src/state_validator.py`
- Modify: `src/keeper.py`
- Modify: `config/system-prompt.txt`
- Modify: `src/state.py`
- Modify: `test_engine.py`

Build tasks

0.1 Define engine-owned fields

Create a canonical list of fields that may not be assigned directly by model output:

- `hp, max_hp`
- `san, max_san`
- `mp`
- `luck`
- `skills`
- `weapon`
- `inventory`
- `location`
- `major_wound`
- `dying`
- `unconscious`
- `temporarily_insane`
- `indefinitely_insane`
- `cthulhu_mythos`
- `checked_skills`
- `front clocks`

- plot point removal
- NPC identity fields

0.2 Replace direct mutation with validation

Current behavior applies arbitrary character fields from `state_delta`.

Replace that with a validator that:

1. Separates allowed proposal fields from engine-owned fields.
2. Rejects or ignores direct writes to engine-owned fields.
3. Logs every rejected key in debug mode.
4. Preserves valid narrative proposals.
5. Never lets a rejected delta crash the turn.

0.3 Route movement through one validated path

Character location changes should only occur through:

- validated `scene_transitions`
- offline movement
- deterministic event results

The destination must exist and be reachable through the current location graph unless the engine explicitly marks the transition as forced.

0.4 Constrain front updates

Temporary Phase 0 behavior may allow bounded front updates for compatibility, but:

- Front IDs must already exist.
- Values must clamp to `0..max`.
- The update should be stored as an increment or validated assignment.
- Phase 3 will replace model-driven clock changes with trigger-driven changes.

0.5 Quick ammunition persistence fix

Before the full item registry lands, prevent the equip/unequip cycle from restoring ammunition.

Temporary approach:

- Track weapon instances by owner and weapon identity.
- Re-equipping retrieves the existing instance rather than creating a fresh catalog copy.
- The full fix arrives in Phase 1.

0.6 Update the LLM output contract

Move the model from direct `state_delta` authority toward:

```
{
  "narration": "...",
  "private_narrations": {},
  "proposed_facts": [],
  "proposed_consequences": [],
  "npc_reactions": [],
```

```
"required_actions": "...",
"dice_requests": []
}
```

For compatibility, Phase 0 may continue accepting legacy `state_delta`, but only after validation.

Acceptance tests

- LLM cannot directly set a character's HP.
- LLM cannot directly move a character by writing `characters.<id>.location`.
- LLM cannot overwrite inventory or equipped weapons.
- Invalid scene transitions are rejected.
- Front clocks cannot exceed configured maximums.
- Rejected fields appear in debug logs.
- A shotgun fired, unequipped, and re-equipped retains its current ammunition.
- Existing saves still load after the validator is introduced.

Exit criteria

The engine can receive malformed, hallucinated, or hostile `state_delta` content without corrupting campaign truth.

Phase 1 — Canonical Items and Objects

Target: v2.8.0

Theme: Gear, clues, keys, weapons, and room objects become persistent entities.

Goal

Replace inventory-as-strings with an item registry. Every meaningful thing in the campaign has an identity, owner, location, state, and history.

This phase turns inventory from a convenience command into part of the campaign engine.

Primary modules

- New: `src/items.py`
- New: `src/objects.py` or a combined `src/entities.py`
- Modify: `src/character.py`
- Modify: `src/keeper.py`
- Modify: `src/combat.py`
- Modify: `src/state.py`
- Modify: scenario schema documentation
- Modify: `test_engine.py`

Core data model

```
@dataclass
class ItemTemplate:
```

```

    id: str
    name: str
    item_type: str
    tags: List[str]
    default_state: Dict[str, Any]

@dataclass
class ItemInstance:
    id: str
    template_id: str
    name: str
    item_type: str
    owner_id: Optional[str]
    location_id: Optional[str]
    quantity: int = 1
    ammunition: Optional[int] = None
    condition: str = "intact"
    tags: List[str] = field(default_factory=list)
    state: Dict[str, Any] = field(default_factory=dict)

```

Characters store item IDs rather than weapon-name strings:

```

inventory: List[str]
equipped_item_id: Optional[str]

```

Build tasks

1.1 Item templates and instances

Separate reusable catalog definitions from campaign-specific instances.

Examples:

- Template: 12_gauge_shotgun
- Instance: item_eleanor_shotgun_001

Two investigators may carry the same model of shotgun while maintaining separate ammunition and condition.

1.2 Inventory migration

Migrate v2.7.x saves:

- Existing weapon names become item instances.
- Equipped weapons map to their corresponding inventory instance.
- Missing inventory entries are seeded from the equipped weapon.
- Duplicate names receive unique instance IDs.

1.3 Item ownership and location

Every item instance must be in exactly one of these states:

- carried by a character
- equipped by a character

- located in a room
- inside a container
- held by an NPC
- destroyed
- removed from play

1.4 Expand item types

Support at minimum:

- weapons
- ammunition
- keys
- clue items
- documents
- light sources
- consumables
- tools
- containers
- valuables
- Mythos artifacts

1.5 Persistent equipment commands

Implement or upgrade:

- inventory
- equip <item>
- unequip
- take <item>
- drop <item>
- give <item> to <character>
- reload
- open <container>
- look at <item>

use <item> may initially route to a deterministic handler for known items and an LLM interpretation path for unusual use.

1.6 Room objects

Create persistent object records for meaningful scenery:

```
@dataclass
class WorldObject:
    id: str
    name: str
    location_id: str
    object_type: str
    state: str
    tags: List[str]
```

```
properties: Dict[str, Any]
```

Examples:

- chapel_nave_door: intact / broken / blocked
- vestry_cabinet: locked / unlocked / open / empty
- ritual_altar: dormant / active / destroyed

1.7 Combat integration

Combat must consume ammunition from the equipped item instance, not from a temporary catalog copy.

Weapon malfunctions should update condition:

- intact
- jammed
- damaged
- destroyed

Acceptance tests

- Two identical shotguns retain separate ammunition.
- Ammunition persists through unequip/equip.
- Ammunition persists through save/load.
- A jammed weapon remains jammed until repaired or cleared.
- Taking an item removes it from the room and adds it to the character.
- Dropping an item places it in the current room.
- Giving an item changes ownership without duplication.
- Clue items can be carried and inspected.
- Scenario-defined objects load with persistent state.
- Save/load round-trips every item and object.

Exit criteria

No important physical truth exists only as a name in a list or as prose in the chronicle.

Phase 2 — Room Truth and Offline Movement

Target: v2.8.1

Theme: The campaign map stops redecorating itself.

Goal

Rooms have stable authored descriptions, persistent state overlays, validated exits, and deterministic movement. Movement through ordinary space should not consume an LLM turn.

Primary modules

- New: `src/room_view.py`
- Modify: `src/spatial.py`

- Modify: `src/keeper.py`
- Modify: `src/state.py`
- Modify: scenario schema and shipped scenarios

Scenario schema additions

Each location gains:

```
{
  "description": "Stable room description.",
  "first_visit": "Optional first-entry description.",
  "revisit": "Optional shorter return description.",
  "details": {
    "altar": "Detail shown when inspected.",
    "pews": "Detail shown when inspected."
  },
  "objects": ["chapel_nave_door", "altar"],
  "lighting": "dim",
  "tags": ["indoor", "ritual", "mythos"]
}
```

Build tasks

2.1 Static base descriptions

The engine serves stable descriptions from scenario data instead of asking the LLM to invent the room again.

2.2 Dynamic room overlays

A room view combines:

- static description
- object state
- visible items
- visible characters
- discovered clues
- recent events
- current front effects
- lighting and weather
- active sensory conditions

Example output:

```
The nave still smells of wet tallow and cold incense.
The west door hangs broken from its hinges.
Sister Amelia stands near the altar, watching the stairs.
Exits: chapel hill, vestry, crypt.
```

2.3 Zero-LLM movement

High-confidence movement declarations resolve locally:

- go to the vestry
- enter the crypt
- head outside
- return to the nave

The engine validates:

- destination exists
- exit is connected
- exit is not blocked or locked
- travel time applies
- movement noise is generated
- occupants update
- current scene updates when appropriate

2.4 Hybrid narration rule

No LLM call is required when entering an ordinary stable room.

An LLM call is still appropriate when entry triggers:

- NPC interaction
- clue revelation
- SAN pressure
- active front event
- trap or hazard
- combat
- major object-state reveal
- private perception
- an unresolved timeline event

The kitchen does not need a costly soliloquy every time. The kitchen after a shotgun blast might.

2.5 Exit state

Connections gain persistent state:

- open
- closed
- locked
- blocked
- hidden
- destroyed
- one-way

The object and exit states must agree. A destroyed door should no longer behave as a locked obstacle.

Acceptance tests

- Ordinary movement produces no LLM timing row.
- Invalid movement lists valid exits without calling the LLM.
- Locked exits block movement.
- Broken doors remain broken after save/load.

- Room descriptions remain stable across turns.
- Dynamic overlays update when objects, occupants, or clues change.
- First-visit and revisit descriptions differ when configured.
- Movement generates the correct noise event hook.

Exit criteria

The map is authoritative, stable, and cheap to traverse, while the LLM remains responsible for dramatic interpretation.

Phase 3 — Event and Trigger Engine

Target: v2.8.2

Theme: The campaign reacts because the engine says it reacts.

Goal

Create a deterministic event pipeline. Important occurrences become append-only events that can activate timeline entries, front thresholds, clue logic, NPC awareness, room changes, and delayed consequences.

This is the missing nervous system between the current engine systems.

Primary modules

- New: `src/events.py`
- New: `src/triggers.py`
- Modify: `src/keeper.py`
- Modify: `src/spatial.py`
- Modify: `src/state.py`
- Modify: `test_engine.py`

Core event model

```
@dataclass
class WorldEvent:
    id: str
    turn: int
    event_type: str
    source_id: Optional[str]
    location_id: Optional[str]
    payload: Dict[str, Any]
    processed_by: List[str] = field(default_factory=list)
```

Initial event types:

- `turn_advanced`
- `skill_result`
- `combat_result`
- `sanity_result`

- noise
- visual
- movement
- object_state_changed
- item_transferred
- front_clock_changed
- clue_discovered
- npc_alerted
- timeline_due

Build tasks

3.1 Append-only event log

Every important engine outcome becomes an event.

The event log should be stored separately from the world snapshot:

```
saves/<scenario>/events.jsonl
saves/<scenario>/world-state.json
```

3.2 Trigger dispatcher

A trigger definition contains:

- trigger ID
- event type
- conditions
- effect
- once/repeat policy
- enabled state

Example:

```
{
  "id": "chapel_bell_turn_10",
  "on": "turn_advanced",
  "conditions": {"turn": 10},
  "effect": {"emit": "chapel_bell_rings"},
  "once": true
}
```

3.3 Timeline activation

Scenario timeline entries become trigger-backed events.

At turn 10, the chapel bell should actually enter the event pipeline instead of merely existing in scenario data.

3.4 Front threshold activation

When a front clock changes, the engine checks its configured thresholds.

At the threshold, it emits the associated event exactly once unless configured otherwise.

3.5 Noise propagation pipeline

Noise flows through the spatial engine:

```
Cause → noise event → propagation → listeners → reactions
```

A gunshot can therefore cause:

- nearby NPC alert
- distant NPC movement
- player-facing audible descriptions
- front advancement
- investigation objectives
- delayed police response

Noise should not directly mean “front +1.” It should be an event that systems may react to.

3.6 Idempotence

A trigger must not fire twice because:

- the same event is processed again
- a save is reloaded
- a retry occurs
- a turn is re-declared after LLM failure

Acceptance tests

- Turn 10 timeline event fires exactly once.
- Front threshold event fires exactly once.
- A gunshot propagates according to room connections.
- A muffled sound is recorded differently from a clear sound.
- Trigger effects survive save/load without repeating.
- Object state changes emit events.
- LLM retry does not duplicate event effects.
- Event log can be inspected turn by turn.

Exit criteria

The campaign has a deterministic cause-and-effect backbone. Timeline, fronts, noise, and object changes no longer depend on the LLM remembering them.

Phase 4 — Facts and Knowledge

Target: v2.8.3

Theme: The world knows the truth; characters know only what they have learned.

Goal

Separate canonical facts from per-character knowledge, false beliefs, rumors, and secrets.

This is essential for both single-player consistency and future multiplayer information hygiene.

Primary modules

- New: `src/facts.py`
- New: `src/knowledge.py`
- Modify: `src/keeper.py`
- Modify: `src/state.py`
- Modify: prompt compilation path

Fact model

```
@dataclass
class Fact:
    id: str
    turn_learned: int
    subject: str
    predicate: str
    value: Any
    fact_type: str      # truth, observation, rumor, lie, belief
    source_id: Optional[str]
    location_id: Optional[str]
    known_by: Set[str]
    evidence: List[str]
    confidence: float = 1.0
    visibility: str = "public"
```

Examples:

- `chapel_crypt_door is_broken true`
- `sister_amelia knows choirmaster_missing`
- `lusk believes investigators_are_police`
- `eleanor observed candles_are_rendered_from_the_missing`

Build tasks

4.1 Fact registry

Canonical facts receive stable IDs and are stored in the save.

4.2 Knowledge grants

Facts can be learned through:

- direct observation
- successful skill checks
- clue discovery
- NPC dialogue
- documents
- being told by another character
- inference granted by the Keeper
- false information or failed Psychology checks

4.3 Beliefs and lies

Not all facts are true.

Support:

- truth
- observation
- rumor
- lie
- false_belief
- delusion

The engine tracks what a character believes without allowing that belief to overwrite world truth.

4.4 Prompt relevance filtering

Do not inject the whole fact ledger into every turn.

Retrieve facts relevant to:

- current location
- visible characters
- active objectives
- current declaration
- current clue chain
- current NPC knowledge
- player's personal knowledge

4.5 Spoiler-safe private views

Private narration should be generated only from facts the character knows, senses, suspects, or misbelieves.

NPC secrets remain Keeper-view unless deliberately revealed.

Acceptance tests

- A fact can exist without every character knowing it.
- Learning a fact updates only the relevant character.
- A false belief does not alter canonical world truth.
- NPC secrets do not appear in player-facing prompts or narration.
- Relevant facts are included in prompts without dumping the full ledger.
- Fact knowledge survives save/load.
- A rumor and a confirmed truth can coexist.

Exit criteria

The campaign can distinguish what happened, who saw it, who was told, who misunderstood it, and who is lying about it.

Phase 5 — Clues and Objectives

Target: v2.8.4

Theme: Investigation becomes earned, connected, and persistent.

Goal

Scenario clues stop being passive data. They become deterministic records with discovery conditions, ownership, relationships, and downstream objective effects.

Primary modules

- New: `src/clues.py`
- New: `src/objectives.py`
- Modify: `src/keeper.py`
- Modify: `src/items.py`
- Modify: `src/facts.py`
- Modify: scenario schema

Clue lifecycle

Each clue moves through states:

1. `hidden`
2. `available`
3. `noticed`
4. `discovered`
5. `interpreted`
6. `used`
7. `missed or destroyed`, when applicable

A clue record tracks:

- clue ID
- location or item
- required skill
- difficulty
- discovery trigger
- found by
- known by
- related facts
- related objectives
- plot point
- red herring status

Build tasks

5.1 Deterministic clue triggers

A clue is discovered when the engine confirms:

- the character is in the right location

- the action targets the relevant object/area
- the required roll succeeds
- prerequisites are met
- the clue has not already been resolved

5.2 Clue-to-item integration

Some clues are physical items and enter the item registry.

Examples:

- ledger
- hymnbook
- whistle
- key
- photograph

5.3 Clue-to-fact integration

Some clues grant facts rather than objects.

Example:

```
Discovered: Tallow Ledger
→ grants fact candles_are_rendered_from_the_missing
→ known_by: eleanor
→ advances objective find_the_choirmaster
```

5.4 Objective registry

Objectives represent the campaign's active dramatic questions.

```
@dataclass
class Objective:
    id: str
    title: str
    owner_type: str          # party, character, npc, front
    owner_id: str
    status: str              # unknown, active, blocked, completed, failed
    related_clues: List[str]
    related_locations: List[str]
    related_characters: List[str]
    urgency: int
    notes: List[str]
```

5.5 Objective board

Players can inspect their current objective state locally:

```
objectives
```

This should display known objectives, not hidden Keeper secrets.

5.6 Objective-driven prompt context

The LLM receives only the relevant objective slice:

- active party objective
- related known clues
- related visible NPCs
- current location links
- recent objective changes

Acceptance tests

- A successful Spot Hidden in the correct room discovers the intended clue.
- A successful roll in the wrong room does not discover it.
- A clue cannot be discovered twice.
- Clue knowledge is per-character until shared.
- Physical clues enter inventory when taken.
- Clue discovery grants the correct facts.
- Objectives update from clue events.
- The objective board shows only player-known information.
- Save/load preserves clue and objective state.

Exit criteria

Investigation progress becomes deterministic and auditable. Clues are earned through location, action, and dice rather than donated by prose.

Phase 6 — NPC Director

Target: v2.8.5

Theme: NPCs act from persistent intent rather than narrative convenience.

Goal

NPCs receive a lightweight deterministic brain. The engine decides what they are trying to do, where they are going, what they know, and how alert they are. The LLM voices their behavior.

Primary modules

- New: `src/npc_director.py`
- Modify: `src/character.py`
- Modify: `src/events.py`
- Modify: `src/spatial.py`
- Modify: `src/facts.py`
- Modify: `src/objectives.py`

NPC brain model

```
@dataclass
```

```

class NPCBrain:
    state: str = "idle"
    objective_id: Optional[str] = None
    target_character_id: Optional[str] = None
    target_location_id: Optional[str] = None
    route: List[str] = field(default_factory=list)
    alert_level: int = 0
    suspicion: Dict[str, int] = field(default_factory=dict)
    attitude: Dict[str, int] = field(default_factory=dict)
    known_facts: Set[str] = field(default_factory=set)
    last_seen: Dict[str, Dict[str, Any]] = field(default_factory=dict)

```

Possible NPC states:

- idle
- patrol
- guard
- suspicious
- investigating
- following
- hiding
- fleeing
- hostile
- performing_ritual
- waiting

Build tasks

6.1 NPC objectives

NPCs receive scenario-defined objectives:

- guard the crypt
- keep investigators away from the ledger
- follow the party
- hide evidence
- protect Sister Amelia
- complete the ritual
- flee if the front reaches a threshold

6.2 Event reactions

NPC brains consume events:

- noise heard → alert increases
- player seen → suspicion changes
- clue taken → objective updates
- front threshold crossed → flee or attack
- trusted character speaks → attitude improves
- lie succeeds → false belief gained

6.3 Deterministic movement

NPC movement uses the same location graph as players.

The engine chooses routes and travel timing. The LLM narrates arrivals, interruptions, and dialogue.

6.4 Awareness and memory

NPCs should remember:

- who they saw
- where they saw them
- what they heard
- which facts they know
- which false beliefs they hold

6.5 Relationship tracking

Track attitude and suspicion per character, not as one generic string.

Example:

```
{
  "eleanor": {"attitude": -2, "suspicion": 4},
  "samuel": {"attitude": 1, "suspicion": 1}
}
```

6.6 LLM voice boundary

The engine provides:

```
NPC: Elias Lusk
State: suspicious
Objective: prevent_escape
Knowledge: heard gunshot, saw Eleanor near vestry
Attitude: hostile but controlled
Next action: intercept at stairs
```

The LLM turns that into dialogue and behavior without changing the underlying intent.

Acceptance tests

- NPC objectives load from scenario data.
- Noise changes NPC alert only if the NPC can hear it.
- NPC movement follows valid graph routes.
- NPCs do not teleport without an explicit event.
- NPC knowledge is separate from player knowledge.
- Attitude and suspicion vary by character.
- Front thresholds can change NPC objectives.
- Save/load preserves NPC brain state.
- The LLM cannot silently change an NPC's objective.

Exit criteria

NPCs behave like persistent inhabitants of the campaign rather than scene decorations summoned by narration.

Phase 7 — Action Grammar and Adjudication

Target: v2.8.6

Theme: Freeform input remains, but common intent becomes mechanically legible.

Goal

Parse player declarations into structured actions before the LLM sees them. Resolve high-confidence actions locally. Preserve freeform fallback for unusual declarations.

This must not turn the game into a strict command parser. Players should still be able to speak naturally.

Primary modules

- New: `src/action_grammar.py`
- Modify: `src/keeper.py`
- Modify: `src/items.py`
- Modify: `src/objects.py`
- Modify: `src/spatial.py`
- Modify: `src/combat.py`

Structured action model

```
@dataclass
class ParsedAction:
    raw_text: str
    intent: str
    actor_id: str
    target_type: Optional[str]
    target_id: Optional[str]
    tool_item_id: Optional[str]
    modifiers: List[str]
    confidence: float
    mechanics: Dict[str, Any]
```

Intent categories

- move
- investigate
- listen
- talk
- attack
- force
- hide

- flee
- use
- take
- drop
- give
- assist
- wait
- custom

Build tasks

7.1 High-confidence deterministic path

Examples:

- go to the vestry
- take the ledger
- equip the shotgun
- shoot the verger
- kick the door in
- search the altar

These should resolve against the world graph, item registry, object state, and skill engine.

7.2 Modifier interpretation

Modifiers adjust mechanics where appropriate:

- carefully → possible bonus die
- quietly → Stealth or reduced noise
- recklessly → penalty or increased noise
- quickly → reduced time but increased risk
- aggressively → intimidation or combat cue

7.3 Ambiguity handling

Low-confidence input should not be forced into mechanics.

Possible outcomes:

1. Ask a short clarifying question.
2. Route to the LLM as a custom declaration.
3. Offer interpretations without meta risk labels.

7.4 Pre-roll integration

The parser feeds the existing preroll system rather than replacing it with prose interpretation.

7.5 Outcome packet generation

Resolved actions produce a structured outcome:

```
{
  "action": "force_object",
```

```
"actor": "eleanor",
"object": "chapel_nave_door",
"skill": "STR",
"roll": 22,
"level": "Hard",
"result": "door_broken",
"events": ["object_state_changed", "noise"]
}
```

The LLM narrates this packet instead of inventing the result.

Acceptance tests

- Common movement, search, combat, and item commands parse reliably.
- Ambiguous input does not trigger the wrong skill.
- Parser confidence controls deterministic versus LLM fallback.
- Parsed actions resolve against actual inventory and object state.
- Failed parsing never consumes a turn incorrectly.
- The freeform TTRPG experience remains available.

Exit criteria

The engine understands common declarations mechanically without forcing players into MUD syntax.

Phase 8 — Truth-to-Voice Prompt Compiler

Target: v2.8.7

Theme: The LLM receives a controlled outcome packet, not the keys to the world.

Goal

Replace broad prompt assembly with purpose-built contexts:

- Keeper context
- Scene context
- Player context
- NPC context
- Outcome context

The model receives enough truth to narrate accurately, but not enough authority to mutate the campaign.

Primary modules

- New: `src/prompt_compiler.py`
- Modify: `src/keeper.py`
- Modify: `src/llm_client.py`
- Modify: `config/system-prompt.txt`

Prompt structure

A typical call should become:

```
CAMPAIGN CONTEXT:
- scenario, era, tone, active fronts, director pressure

SCENE TRUTH:
- current location, static room state, objects, occupants, exits

ACTOR TRUTH:
- acting character, relevant sheet slice, equipment, condition

OUTCOME PACKET:
- parsed action, dice result, mechanical consequence, events fired

RELEVANT KNOWLEDGE:
- facts known by the actor
- NPC knowledge relevant to the scene
- clue state visible to the actor

VOICE TASK:
Narrate the outcome. Do not alter mechanics. Do not reveal hidden facts.
```

Build tasks

8.1 Context separation

Create distinct context builders:

- build_keeper_context
- build_scene_context
- build_player_context
- build_npc_context
- build_outcome_context

8.2 Tiered prompt sizes

- Minimal: routine local resolution
- Standard: normal scene turn
- Full: cinematic or socially complex turn
- Heavy: major set piece only

8.3 Static scene caching

Stable room descriptions do not need to be regenerated each turn. Cache them until room state changes.

8.4 Proposal-only output

The LLM returns:

- narration
- private narrations

- dialogue
- proposed facts
- proposed consequences
- dice requests
- optional mode switch

The validator translates proposals into engine-safe changes.

8.5 Spoiler-safe contexts

NPC secrets, hidden clues, and unknown facts must not leak into player-facing prompts unless the Keeper deliberately reveals them.

Acceptance tests

- The same deterministic outcome produces the same core prompt packet.
- Hidden facts do not appear in player context.
- NPC knowledge appears only in NPC/Keeper context.
- Prompt size stays within target tiers.
- The LLM cannot return direct state mutation fields.
- Narration accurately reflects dice and event outcomes.
- Cached room descriptions invalidate only on state change.

Exit criteria

The LLM is no longer a state database with a narration habit. It is a voice layer over deterministic campaign truth.

Phase 9 — Persistence, Replay, and Audit

Target: v2.8.8

Theme: A campaign can be reconstructed, inspected, and trusted.

Goal

The campaign state becomes replayable from snapshots plus append-only events. This supports debugging, regression tests, multiplayer reconnects, and long-running campaigns.

Primary modules

- New: `src/replay.py`
- New: `src/audit.py`
- Modify: `src/state.py`
- Modify: `src/events.py`
- Modify: save layout

Build tasks

9.1 Snapshot and event separation

Use:

```
saves/<scenario>/world-state.json  
saves/<scenario>/events.jsonl  
saves/<scenario>/snapshots/turn-0020.json
```

9.2 Event replay

A campaign can be reconstructed by:

1. Loading the initial scenario state.
2. Replaying events through the deterministic engines.
3. Verifying that the resulting world matches the snapshot.

9.3 Campaign audit command

Add a local audit mode:

```
python -m src.main --audit
```

It should report:

- orphan items
- characters in invalid locations
- broken references
- duplicate event IDs
- repeated trigger firings
- impossible HP/SAN values
- clues granted without discovery
- NPCs with invalid objectives
- facts known by nonexistent characters

9.4 Deterministic procedural seeds

Where procedural behavior exists, all randomness should derive from:

```
campaign_seed + turn + event_type + salt
```

9.5 Save migrations

Every schema change receives a migration path and tests using older saves.

Acceptance tests

- Event replay reproduces the current snapshot.
- Save/load round-trips all registries.
- Audit detects intentionally corrupted state.
- Events do not duplicate after reload.
- Procedural draws are deterministic for the same seed.

- Old v2.7.x saves migrate successfully.

Exit criteria

The campaign engine can explain exactly how the world reached its current state.

Phase 10 — Cost and Provider Hardening

Target: v2.8.9

Theme: Reduce hidden reasoning and preserve provider independence.

Goal

Now that the engine removes unnecessary LLM turns, optimize the remaining calls without sacrificing Keeper quality.

Primary modules

- Modify: `src/llm_client.py`
- Modify: `src/prompt_compiler.py`
- New: `config/system-prompt-small.txt`
- Modify: `test_latency.py`

Build tasks

10.1 Reasoning-budget A/B testing

Measure current settings against low and medium reasoning configurations.

Track:

- cost
- latency
- visible output length
- hidden reasoning share
- malformed JSON rate
- retry rate
- dice request accuracy
- narration quality
- state consistency

10.2 Tiered model routing

Route calls by need:

- routine narration → default model
- major social scene → standard or heavy
- combat aftermath → standard
- cinematic set piece → heavy
- mechanical movement → no model

10.3 Small system prompt

Create a short prompt for local or weaker models:

- core rules
- output schema
- three examples
- strict no-state-authority rule

10.4 Provider degradation

Unsupported parameters must no-op safely rather than kill a turn.

10.5 Circuit breakers

If malformed output or latency exceeds configured thresholds:

- fall back to smaller prompt
- fall back to default tier
- preserve the turn
- keep the session alive

Acceptance tests

- Reasoning-budget settings measurably reduce completion cost where supported.
- Unsupported provider parameters do not crash.
- Local models receive the small prompt.
- Heavy tier is used only for configured escalation categories.
- LLM failure still refunds the turn.
- No movement-only turn creates an API timing row.

Exit criteria

The engine spends LLM budget only on scenes that benefit from an LLM.

Phase 11 — v3.0 Multiplayer Campaign Server

Target: v3.0

Theme: Many players, one canonical world.

Goal

Build multiplayer on top of the completed truth engine, not in parallel with it.

Primary modules

- New: `src/turn_manager.py`
- New: `src/view_compiler.py`
- New: `src/event_bus.py`
- New: `src/terminal_server.py`

- New: `src/delta_sync.py`
- Modify: `src/keeper.py`
- Modify: `src/prompt_compiler.py`

Revised v3 principles

Per-player views

Each player receives only what their character perceives and knows.

Per-scene LLM calls

Resolve one scene at a time rather than one paid LLM call per player whenever possible.

Deterministic NPCs

NPC autonomy remains state-machine and objective-driven. The LLM voices NPCs but does not decide their canonical movement or knowledge.

Event synchronization

Cross-scene effects propagate through the event system from Phase 3.

Reconnect through deltas

Reconnecting players receive only events and changes their character would have perceived.

Build order

1. `turn_manager.py`
2. `view_compiler.py`
3. `event_bus.py`
4. `terminal_server.py`
5. `delta_sync.py`
6. multiplayer state persistence
7. multiplayer audit tools

Acceptance tests

- Eight players can connect without blocking one another.
- A gunshot in one scene propagates correctly to another.
- Players receive different views based on location and knowledge.
- NPC secrets never leak into unrelated player views.
- A disconnected player reconnects with a correct delta summary.
- One scene can run combat pacing while another remains in exploration pacing.
- Save/load preserves every player's knowledge and position.

Exit criteria

Multiple players share one persistent, auditable campaign world without the LLM becoming the synchronization layer.

4. Scenario Schema Evolution

The scenario format should evolve alongside the engine.

Location schema

```
{
  "name": "Nave of the Tallow Chapel",
  "description": "...",
  "first_visit": "...",
  "revisit": "...",
  "details": {},
  "objects": [],
  "connections": {},
  "sound_propagation": {},
  "line_of_sight": [],
  "lighting": "dim",
  "tags": []
}
```

NPC schema

```
{
  "id": "elias_lusk",
  "name": "Elias Lusk",
  "characteristics": {},
  "skills": {},
  "location": "chapel_nave",
  "brain": {
    "state": "suspicious",
    "objective": "prevent_escape",
    "alert_level": 1,
    "relationships": {},
    "known_facts": []
  }
}
```

Object schema

```
{
  "id": "chapel_nave_door",
  "name": "Nave Door",
  "location": "chapel_nave",
  "object_type": "door",
  "state": "locked",
  "properties": {
    "break_skill": "STR",
    "break_difficulty": "Regular",
    "noise_on_force": 3
  }
}
```

Clue schema

```
{
  "id": "tallow_ledger",
  "name": "The Tallow Ledger",
  "location": "chapel_vestry",
  "skill": "Library_Use",
  "difficulty": "Regular",
  "state": "hidden",
  "grants_facts": ["candles_are_rendered_from_the_missing"],
  "advances_objectives": ["find_the_choirmaster"],
  "item_template": "document_ledger"
}
```

Objective schema

```
{
  "id": "find_the_choirmaster",
  "title": "Find the Choirmaster",
  "owner_type": "party",
  "status": "active",
  "related_clues": ["tallow_ledger", "wax_sealed_hymnbook"],
  "related_locations": ["chapel_vestry", "chapel_crypt"],
  "urgency": 3
}
```

5. Testing Strategy

Every phase must land with deterministic offline tests before any paid LLM smoke test.

Required test categories

- 1. State authority tests**
Prove the model cannot mutate engine-owned truth.
- 2. Persistence tests**
Save/load every registry and relation.
- 3. Migration tests**
Load saves from the previous schema version.
- 4. Event idempotence tests**
Prove triggers fire exactly once.
- 5. Knowledge isolation tests**
Prove secrets and facts do not leak.
- 6. Spatial tests**
Prove movement, sound, line of sight, and blocked exits behave correctly.
- 7. Inventory tests**
Prove item uniqueness, ownership, condition, and ammunition persistence.

8. NPC behavior tests

Prove alert, objective, route, and relationship changes.

9. Prompt packet tests

Prove prompts reflect outcomes and hide forbidden state.

10. Offline integration tests

Run full sessions in `--mock` mode with scripted declarations.

6. Immediate First Sprint

The first coding sprint should be intentionally small and test-heavy.

Sprint goal

Ship v2.7.6.1: Truth Firewall.

Sprint tasks

1. Add `src/state_validator.py`.
2. Define engine-owned character fields.
3. Wrap `_apply_state_delta()` in validation.
4. Route all movement through validated scene transitions.
5. Clamp front updates and log rejected writes.
6. Add temporary weapon-instance preservation so ammunition does not reset.
7. Add rejected-delta debug logging.
8. Add regression tests for malicious or hallucinated deltas.
9. Run the full offline suite.
10. Play one mock session and one paid smoke session.

Sprint exit

The model can still narrate normally, but it can no longer corrupt the campaign's canonical state.

7. Design Guardrails

Do not make the game a command parser

Natural language remains the main interface. Structured commands exist for reliability, not as a requirement.

Do not dump the entire world into prompts

Prompts should receive selected truth relevant to the current actor, scene, objective, and outcome.

Do not let NPCs become autonomous LLM agents

NPC intent, location, awareness, and knowledge are deterministic. The LLM performs their voice.

Do not optimize prompt text before reducing calls

The current measurements show hidden reasoning dominates cost. Fewer calls and lower reasoning budgets matter more than shaving a few hundred input tokens.

Do not build multiplayer before the truth engine

Multiplayer amplifies every state inconsistency. The single-player engine must become trustworthy first.

Do not let atmosphere overwrite evidence

A great line of narration is not permission to change HP, move an NPC, restore ammunition, or reveal a hidden clue.

8. Final Target Experience

When this roadmap is complete, a turn should work like this:

1. The player declares an action naturally.
2. The action grammar interprets high-confidence intent.
3. The engine validates the action against world truth.
4. Dice resolve risk.
5. Items, objects, locations, clues, facts, NPCs, fronts, and timelines update deterministically.
6. Events propagate through the world.
7. The prompt compiler builds a bounded outcome packet.
8. The LLM narrates the result in the Keeper's voice.
9. The validator accepts only safe proposals.
10. The campaign saves a snapshot and appends the event history.

That is the transition from an impressive AI storytelling loop to a persistent campaign engine with an AI Keeper.